Project Submission

Course Title : Artificial Intelligence Lab

Course Code : CSE – 413

Project Name: Titanic - Machine Learning from Disaster

Submitted To

Md. Israfil Mahmud Raju

Lecturer

Department of CSE

Daffodil International University

Submitted By

- **Thoufiq Zumma ID: 181-15-10968**
- **K.B.M Tahmiduzzaman ID: 181-15-11150**

Section: L

Department of CSE

Daffodil International University

Submission Date: 20.04.2021

Titanic: Machine Learning from Disaster

Titanic was a British passenger liner operated by the White Star Line that sank in the North Atlantic Ocean in the early morning hours of 15 April 1912, after striking an iceberg during her maiden voyage from Southampton to New York City. Of the estimated 2,224 passengers and crew aboard, more than 1,500 died, making the sinking one of modern history's deadliest peacetime commercial marine disasters.

Source Code: https://colab.research.google.com/drive/1P-VrtbP5bvA0_Ggns0WX7TSiafdpgz1C?usp=sharing

So we need to first study of our data

1 - PassengerId - Column does not depend on survive

2 - Pclacss - it's needed for prediction .

3 - Survived

4 - Name - it's not useful for our prediction

5 - Sex - Male/ Female depends on survived because female always get a first chance

6 - Age - Age depends on survivde

7 - SibSp - Having siblings/spouse depends on survived

8 - Parch - Number of childs depends on survived

9 - Ticket - Ticket not create impact on survived

10 - Fare - Fare create impact om survived because who have a costly tickets ,that person have more chance to get first in lifeboat

11 - Cabin - Cabin have more null values and its not create any impact on survived

12 - Embarked - it's create impact on survived

Contents:

1.Import Data

2.Visualizing data

3.Cleaning data

4.Choosing Best Model

1.Import Data:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
sns.set()
```

Loading Dataset:

```
train =pd.read_csv('train.csv')
test=pd.read_csv('test.csv')
```

Looking into the train dataset

```
train.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Total rows and columns (train)

```
train.shape
```

```
(891, 12)
```

Looking into the testing dataset

```
test.head()
```

	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

Total rows and columns (testdata)

```
test.shape
```

```
(418, 11)
```

Now we see that the test data has 418 rows and 11 columns.

Train data rows = 891

Test data rows = 418

Total rows = 891+418 = 1309

We can see that around 2/3 of total data is set as Train data and around 1/3 of total data is set as Test data.

2.Visulazation:

Relationship between Features and Survival

```
survived = train[train['Survived']==1]
not_survived=train[train['Survived']==0]

print("Survived: %i (%.1f%%)"%(len(survived),float(len(survived))/len(train)*100.0))
print(" Not Survived: %i (%.1f%%)"%(len(not_survived),float(len(not_survived))/len(train)*100.0))
print ("Total: %i"%len(train))
```

```
Survived: 342 (38.4%)
 Not Survived: 549 (61.6%)
Total: 891
```

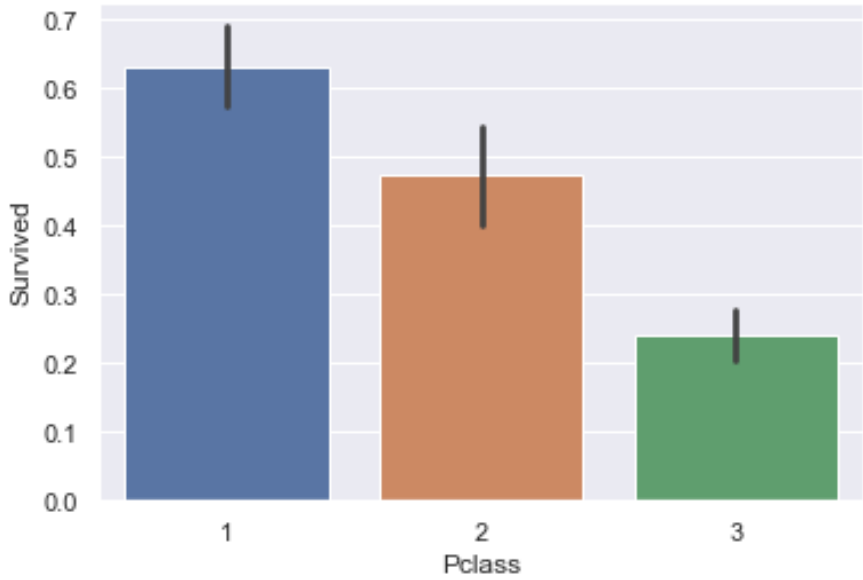
Pclass vs Survival

```
train.Pclass.value_counts()
```

```
3    491
1    216
2    184
Name: Pclass, dtype: int64
```

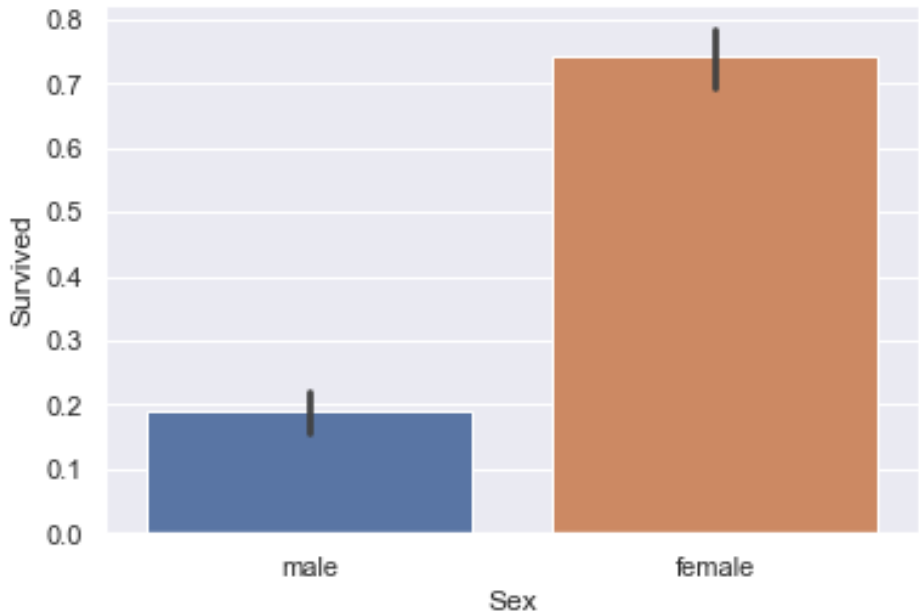
```
sns.barplot(x='Pclass',y='Survived',data=train)
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x17c7800ef48>
```




```
sns.barplot(x='Sex',y='Survived',data=train)
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x17c7877d3c8>
```

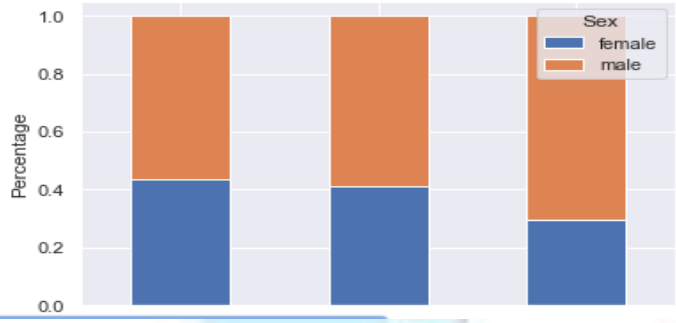


Pclass & Sex vs Survival

```
tab = pd.crosstab(train['Pclass'],train['Sex'])
print(tab)
tab.div(tab.sum(1).astype(float),axis=0).plot(kind="bar",stacked=True)
plt.xlabel('Pclass')
plt.ylabel('Percentage')
```

```
Sex     female  male
Pclass
1           94   122
2           76   108
3          144   347

Text(0, 0.5, 'Percentage')
```

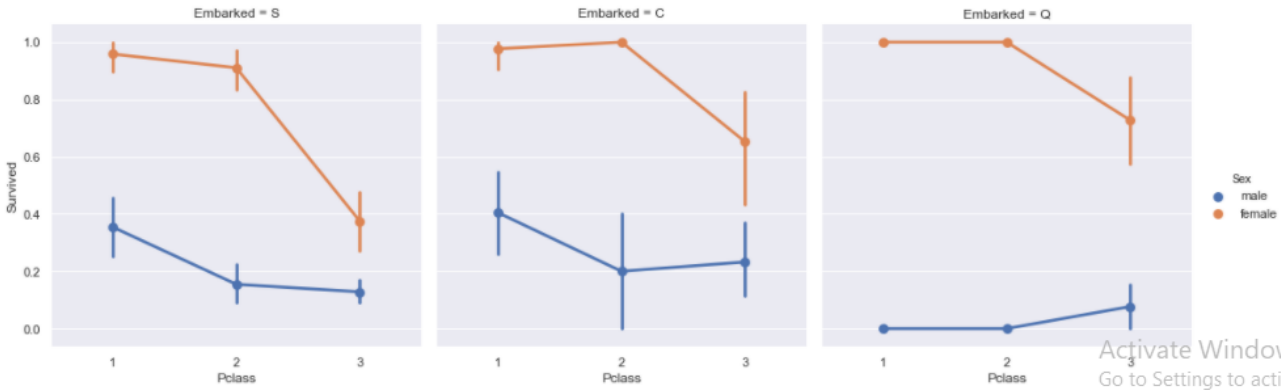


Pclass,Sex & Embarked vs. Survival

```
sns.factorplot(x='Pclass',y='Survived',hue='Sex',col='Embarked',data=train)
```

```
C:\Users\Thoufiq\anaconda3\lib\site-packages\seaborn\categorical.py:3669: UserWarning: The `factorplot` function has been renam
ed to `catplot`. The original name will be removed in a future release. Please update your code. Note that the default `kind` i
n `factorplot` (``point``) has changed ``strip`` in `catplot`.
  warnings.warn(msg)
```

```
<seaborn.axisgrid.FacetGrid at 0x17c78970648>
```



Activate Windows
Go to Settings to activat

From the above plot, it can be seen that:

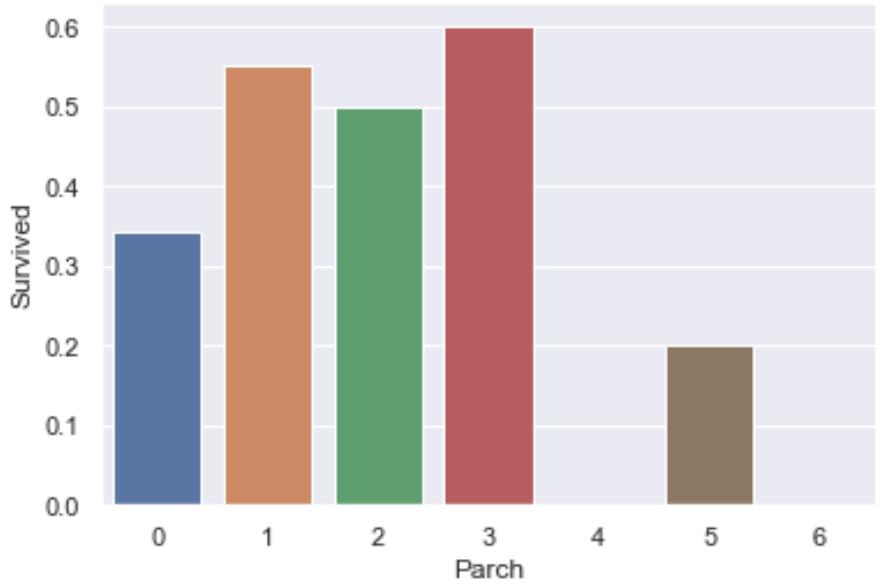
1.Almost all females from Pclass 1 and 2 survived.

2.Females dying were mostly from 3rd Pclass.

3.Males from Pclass 1 only have slightly higher survival chance than Pclass 2 and 3.

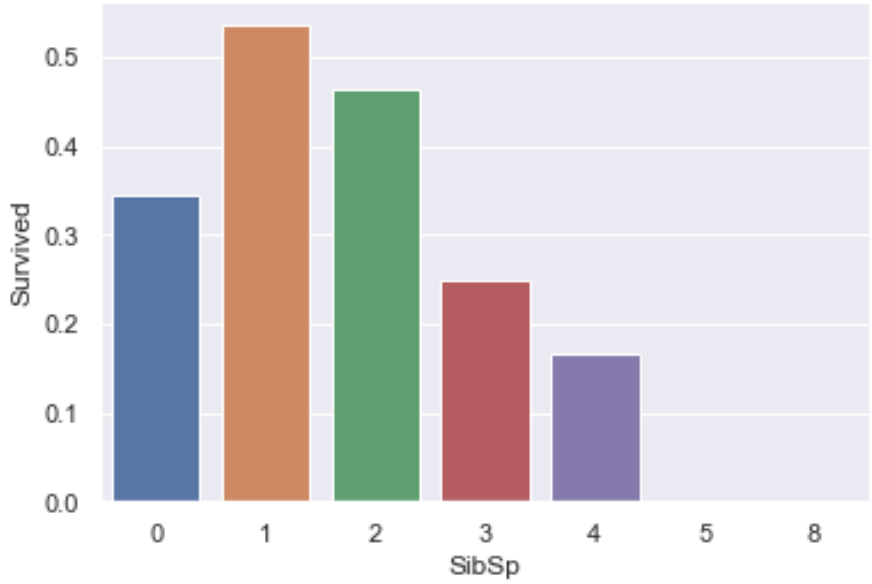
```
sns.barplot(x='Parch',y='Survived',ci=None,data=train)
# ci=None will hide the error bar
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x17c78d34e88>
```



```
sns.barplot(x='SibSp',y='Survived',ci=None,data=train)
#ci=None will hide the error bar
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x17c78da3288>
```



3.Cleaning data:

Feature Extraction

In this section, we select the appropriate features to train our classifier. Here, we create new features based on existing features. We also convert categorical features into numeric form.

Name Feature

Let's first extract titles from Name column.

Sex Feature

We convert the categorical value of Sex into numeric. We represent 0 as female and 1 as male.

Embarked Feature

There are empty values for some rows for Embarked column. The empty values are represented as "nan" in below list.

Age Feature

We first fill the NULL values of Age with a random number between (mean_age - std_age) and (mean_age + std_age).

We then create a new column named AgeBand. This categorizes age into 5 different age range.

Fare Feature

Replace missing Fare values with the median of Fare.

SibSp & Parch Feature

Combining SibSp & Parch feature, we create a new feature named FamilySize.

Feature Selection

We drop unnecessary columns/features and keep only the useful ones for our experiment. Column PassengerId is only dropped from Train set because we need PassengerId in Test set while creating Submission file to Kaggle.

Classification & Accuracy:

There are many classifying algorithms present. Among them, we choose the following Classification algorithms for our problem:

1. Logistic Regression
2. Support Vector Machines (SVC)
3. Linear SVC
4. k-Nearest Neighbor (KNN)
5. Decision Tree
6. Random Forest
7. Naive Bayes (GaussianNB)
8. Perceptron
9. Stochastic Gradient Descent (SGD)

Here's the training and testing procedure:

First, we train these classifiers with our training data.

After that, using the trained classifier, we predict the Survival outcome of test data.

Finally, we calculate the accuracy score (in percentange) of the trained classifier.

Please note: that the accuracy score is generated based on our training dataset.

4.Choosing Best Model:

```
# Importing Classifier Modules
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC, LinearSVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import Perceptron
from sklearn.linear_model import SGDClassifier
```

Logistic Regression

```
]: clf = LogisticRegression()
   clf.fit(X_train,y_train)
   y_pred_log_reg =clf.predict(X_test)
   acc_log_reg = round( clf.score(X_train, y_train) * 100, 2)
   print (str(acc_log_reg) + ' percent')

   80.02 percent
```

Support Vector Machine (SVM)

```
]: clf = SVC()
clf.fit(X_train, y_train)
y_pred_svc = clf.predict(X_test)
acc_svc = round(clf.score(X_train, y_train) * 100, 2)
print (acc_svc)
```

```
83.16
```

Linear SVM ¶

```
clf = LinearSVC()
clf.fit(X_train, y_train)
y_pred_linear_svc = clf.predict(X_test)
acc_linear_svc = round(clf.score(X_train, y_train) * 100, 2)
print (acc_linear_svc)
```

```
78.79
```

Nearest Neighbors

```
clf = KNeighborsClassifier(n_neighbors = 3)
clf.fit(X_train, y_train)
y_pred_knn = clf.predict(X_test)
acc_knn = round(clf.score(X_train, y_train) * 100, 2)
print (acc_knn)
```

```
79.12
```

Decision Tree

```
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)
y_pred_decision_tree = clf.predict(X_test)
acc_decision_tree = round(clf.score(X_train, y_train) * 100, 2)
print (acc_decision_tree)
```

```
87.32
```

Random Forest

```
clf = RandomForestClassifier(n_estimators=100)
clf.fit(X_train, y_train)
y_pred_random_forest = clf.predict(X_test)
acc_random_forest = round(clf.score(X_train, y_train) * 100, 2)
print (acc_random_forest)
```

```
87.32
```

Gaussian Naive Bayes

```
clf = GaussianNB()
clf.fit(X_train, y_train)
y_pred_gnb = clf.predict(X_test)
acc_gnb = round(clf.score(X_train, y_train) * 100, 2)
print (acc_gnb)
```

```
77.22
```

Perceptron

```
clf = Perceptron(max_iter=5, tol=None)
clf.fit(X_train, y_train)
y_pred_perceptron = clf.predict(X_test)
acc_perceptron = round(clf.score(X_train, y_train) * 100, 2)
print (acc_perceptron)
```

```
76.43
```

Stochastic Gradient Descent (SGD)

```
clf = SGDClassifier(max_iter=5, tol=None)
clf.fit(X_train, y_train)
y_pred_sgd = clf.predict(X_test)
acc_sgd = round(clf.score(X_train, y_train) * 100, 2)
print (acc_sgd)
```

```
76.43
```

Comparing Models

Let's compare the accuracy score of all the classifier models used above.

	Model	Score
4	Decision Tree	87.32
5	Random Forest	87.32
1	Support Vector Machines	83.16
0	Logistic Regression	80.02
3	KNN	79.12
2	Linear SVC	78.79
6	Naive Bayes	77.22
7	Perceptron	76.43
8	Stochastic Gradient Decent	76.43

From the above table, we can see that Decision Tree and Random Forest classifiers have the highest accuracy score.

Among these two, we choose Random Forest classifier as it has the ability to limit overfitting as compared to Decision Tree classifier.